

第一页 · Agent 的基本构成

一个 Agent 至少包含三部分：感知（读取输入与环境状态）、决策（决定下一步做什么）、执行（调用工具改变环境或获取信息）。

与单轮问答的根本区别在于「循环」：Agent 会根据执行结果重新决策，直到任务完成或达到轮数上限。

没有循环的不叫 Agent，那只是一次带工具的函数调用。

第二页 · ReAct 模式

ReAct = Reasoning + Acting，把推理和行动交替进行：

```
Thought    模型思考当前该做什么
Action     调用某个工具
Observation 拿到工具返回的结果
... 循环 ...
Final Answer
```

关键在于 Observation 会回填进上下文，影响下一轮的 Thought。
这让模型能够根据中间结果调整策略，而不是一条道走到黑。

实现上有两种：一种是让模型输出结构化文本自己解析，
另一种是用 API 原生的 function calling。后者更稳，
因为参数由模型按 schema 生成，不用自己写解析器。

第三页 · 记忆与上下文管理

A g e n t 的记忆分两类：

短期记忆 —— 当前会话的对话历史。受上下文窗口限制，太长必须截断或压缩。

长期记忆 —— 跨会话保留的知识。通常存进向量库，需要时检索回来，本质上就是 R A G。

短期记忆的压缩策略：最近几轮保留完整过程，更早的只留问题与结论，超预算则从最老的开始淘汰。
依据是：刚发生的事需要细节，久远的事只需要结论。

第四页 · 工具设计的经验

工具的 `description` 是模型判断要不要调用的唯一依据，写得含糊就会该调不调、不该调乱调。

好的 `description` 说清两件事：这个工具做什么，以及什么情况该用、什么情况不该用。

工具数量不宜过多。超过十个之后模型的选择准确率会下降，这时应该考虑分层：先用一个粗粒度工具定位，再用细粒度工具取详情（先粗后精）。

工具返回的内容要控制长度。返回一大段原文会挤爆上下文，更好的做法是返回摘要加一个可深挖的标识。

第五页 · 失败模式与防护

常见的四种失败：

一、死循环。模型反复调用同一个工具。

防护：设置最大轮数，通常 3 到 5 轮。

二、参数非法。模型生成的 J S O N 解析失败。

防护：`try / except` 包住解析，失败时把错误返回给模型。

三、越界发挥。工具返回的是事实，模型据此展开臆测。

防护：在 `P r o m p t` 里明确「只能陈述工具返回的事实」。

四、静默降级。某个工具失败后流程继续，

但结果已经不完整，而用户看不出来。

防护：失败要么明确告知，要么中止，不要假装成功。